

Flujos de CI/CD: GITEA - TRIVY - HARBOR - ARGOCD

Documentación de los tres flujos principales de integración continua y sincronización con ARGOCD.

1. Flujo de Construcción de Imagen y Análisis de Vulnerabilidades

Descripción

Pipeline de GITEA Actions que construye una imagen Docker, analiza vulnerabilidades con Trivy y la sube a Harbor si no hay vulnerabilidades críticas o altas.

Flujo de Ejecución

```

Push a repo GITEA
  |
  v
Trigger Gitea Actions
(on push / PR)
  |
  v
1. Checkout código
  |
  v
2. Build imagen Docker
   tag: harbor/.../app:commit
   |
   v
3. Scan con Trivy
   busca CRITICAL + HIGH
   |
   +----> ¿Vulnerabilidades críticas?
   |
   SI|      NO
   |      |
   v      v
FAIL    4. Login Harbor
(error)  |
         v
         5. Push imagen
           v1.0.0, latest
           |
           v
         6. Pipeline finalizado
           Report en output

```

Detalles de Implementación

Trigger:

- `on: [push]` en ramas main/master

- `on: [pull_request]` a main/master

Pasos:

1. **Checkout:** `actions/checkout@v3`
2. **Build Docker:**
 - Dockerfile en raíz del repo
 - Tag format: `harbor.cnie.internal/project/app:${commit-hash}`
 - Tag adicional: `harbor.cnie.internal/project/app:latest`
3. **Trivy Scan:**
 - Comando: `trivy image --severity CRITICAL,HIGH --format json`
 - Si encuentra vulnerabilidades: `exit 1` (falla el pipeline)
 - Report en output con detalle de vulnerabilidades
4. **Push a Harbor:**
 - Autenticación con secrets de Harbor
 - Push de imagen con tags versionados

Secrets requeridos:

- `REGISTRY_USER` : Usuario de Harbor
- `REGISTRY_PASS` : Contraseña/Token de Harbor
- `REGISTRY_URL` : URL del registry (ej: harbor.cnie.internal)

2. Flujo de Sincronización con ARGOCD

Descripción

ARGOCD sincroniza automáticamente la nueva imagen Docker con la aplicación desplegada. El proceso detecta cambios en el repositorio de configuración y actualiza el deployment.

Flujo de Ejecución

Imagen subida a Harbor

|

v

Update del repo Helm/Kustomize

- values.yaml con nueva imagen
- Commit automático

|

v

ARGOCD detecta cambio

(polling o webhook)

|

v

Compara:

- Git (desired): imagen nueva
- K8s (actual): imagen vieja

|

v

¿Diferente?

|

SI

v

Auto-Sync

Pruning + SelfHeal

|

v

K8s rollout

Rolling update

Health checks

|

v

Nueva imagen corriendo

Old pods mueren

|

v

ARGOCD SYNCED

Status: Healthy
Report en output

3. Flujo de Limpieza de Imágenes Antiguas

Descripción

Pipeline de GITEA Actions que consulta ARGOCD para saber qué imágenes están en uso, obtiene la lista de imágenes de Harbor y elimina las versiones antiguas, manteniendo solo:

1. La versión en uso actualmente
2. La versión más reciente cercana a la versión en uso

Flujo de Ejecución

Trigger: Schedule cron

(cada día a las 02:00 UTC o manual)

|

v

1. Conectar a ARGOCD API

Token + auth

|

v

2. Query apps en ARGOCD

GET /api/v1/applications

|

v

3. Extraer imagen en uso

Parsear deployment YAML

Ej: harbor/app:v1.4.7

|

v

4. Conectar a Harbor API

Auth + token

|

v

5. Listar imágenes Harbor

GET /api/v2.0/projects/*/repos

Orden descendente por fecha

|

v

6. Identificar qué borrar

v1.5.0 (latest) ---> BORRAR

v1.4.9 -----> BORRAR

v1.4.8 -----> BORRAR

v1.4.7 (en uso) ----> MANTENER

v1.4.6 -----> MANTENER

v1.4.5 -----> BORRAR

|

v

7. Ejecutar borrados Harbor

DELETE /api/v2.0/repositories

Log cada eliminación

|

v

8. Generar reporte

- Borradas: N imágenes
- Espacio: X GB
- Mantenidas: 2

|

v

9. Report final en output

Status: SUCCESS o FAILURE

Detalle de acciones

Detalles de Implementación

Trigger:

- Schedule: `0 2 * * *` (2:00 UTC diariamente)
- Manual: `workflow_dispatch`

Pasos principales:

1. Autenticación ARGOCD:

Conectar con token y credenciales

2. Obtener aplicación en ARGOCD:

Query a la API para obtener apps

3. Extraer imagen actual:

Parsear JSON y extraer imagen en uso

4. Listar imágenes en Harbor:

Query a la API de Harbor

5. Lógica de retención:

- Obtener todas las imágenes
- Identificar versión en uso
- Marcar como MANTENER: versión en uso + más reciente cercana
- Marcar resto como BORRAR

6. Eliminar imágenes:

Ejecutar DELETES en Harbor

Secrets requeridos:

- `ARGOCD_USER` : Usuario de ARGOCD
- `ARGOCD_PASS` : Contraseña/Token de ARGOCD
- `ARGOCD_URL` : <http://argocd.cnie.internal>
- `HARBOR_USER` : Usuario de Harbor
- `HARBOR_PASS` : Contraseña/Token de Harbor
- `HARBOR_URL` : harbor.cnie.internal

Consideraciones importantes:

- Mantener siempre 2 versiones mínimo
- No borrar la imagen en uso actualmente
- Ejecutar con permisos suficientes en Harbor
- Logging detallado de cada eliminación
- Manejo de errores en cada paso